SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Week 11: Task C.6

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY – MAR-2026

Contents

1. Overview 3
2. Implementation Summary 3
   2.1 Ensemble components 3
   2.2 Key functions added for Task C.6 3
3. Less Straightforward Code Lines 4
   3.1 SARIMA parameters and fitting 4
   3.2 Aligning DL and SARIMA predictions to the same horizon 4
   3.3 Weighted ensemble and why weights matter 5
   3.4 Metrics used for comparison 5
4. References Used 5
5. Experiment Results Summary (Multistep / Multivariate / Combined) 5
   5.1 Different ensemble configurations (DL type + SARIMA params) 5
   5.2 Best-performing configurations (summary) 6
   5.3 Comparison table (Best config per weight run) 6

1. Overview

In Task C.6 (v0.5), the objective is to improve forecasting robustness by using an ensemble approach that combines:

1. a statistical time-series model (SARIMA via statsmodels) and
2. a deep learning model (LSTM/GRU/RNN from my dynamic builder in Task C.4).

I implemented a workflow that trains the deep learning model on the training set, fits SARIMA on the training Close series, produces forecasts on the same test horizon, and then combines the two predictions using a weighted ensemble:

$$\hat{y}_{ens} = w \cdot \hat{y}_{DL} + (1 - w) \cdot \hat{y}_{SARIMA}$$

I then ran multiple experiments across different DL model types, SARIMA hyperparameters, and ensemble weights.

2. Implementation Summary

2.1 Ensemble components

- Deep Learning (DL): built using my dynamic function build_recurrent_model(layer_name, num_layers, units, ...) so I can swap between LSTM / GRU / RNN without rewriting training code.
- SARIMA: fitted using SARIMAX from statsmodels, which supports both ARIMA and seasonal ARIMA through (p,d,q) and (P,D,Q,m) parameters.
- Ensemble: computed using a weighted average controlled by w_dl

2.2 Key functions added for Task C.6

I implemented these helper functions:

1. fit_sarima(train_close, order, seasonal_order) — fit SARIMA model on training Close series

```python
def fit_sarima(train_series: pd.Series, order=(1, 1, 1), seasonal_order=(1, 0, 1, 5)):
    """
    Fit SARIMA on a 1D training series and return the fitted model.
    """
    model = SARIMAX(
        train_series,
        order=order,
        seasonal_order=seasonal_order,
        enforce_stationarity=False,
        enforce_invertibility=False
    )
    fitted = model.fit(disp=False)
    return fitted
```

2. forecast_sarima(fitted_model, steps) — forecast the test horizon length

```python
def forecast_sarima(fitted_model, steps):
    """Forecast next 'steps' points."""
    return fitted_model.forecast(steps=steps)
```

3. ensemble_weighted(pred_dl, pred_sarima, w_dl) — combine predictions

```python
def ensemble_weighted(pred_dl, pred_sarima, w_dl = 0.2):
    """
    Weighted ensemble:
      y_hat = w_dl * DL + (1-w_dl) * SARIMA
    """
    pred_dl = np.asarray(pred_dl).reshape(-1)
    pred_sarima = np.asarray(pred_sarima).reshape(-1)
    return w_dl * pred_dl + (1 - w_dl) * pred_sarima
```

4. regression_metrics(y_true, y_pred) — compute MAE/MSE/RMSE consistently across models.

```python
def regression_metrics(y_true, y_pred):
    y_true = np.asarray(y_true).reshape(-1)
    y_pred = np.asarray(y_pred).reshape(-1)
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    return mae, mse, rmse
```

3. Less Straightforward Code Lines

3.1 SARIMA parameters and fitting

Non-trivial part: understanding order=(p,d,q) and seasonal_order=(P,D,Q,m) and choosing a reasonable m.

- order=(p,d,q) controls the autoregressive terms, differencing, and moving average terms.
- seasonal_order=(P,D,Q,m) controls the seasonal AR/DIFF/MA behavior and the seasonal period length m.
- I used m=5 to represent a weekly trading cycle (5 trading days).

Why fit only on training:

To avoid data leakage, SARIMA is fitted only on the training Close series and then used to forecast the test horizon.

3.2 Aligning DL and SARIMA predictions to the same horizon

A key requirement for ensembling is that both models forecast the same timestamps and output vectors of the same length. In my implementation:

- DL produces one prediction per test sample (single-step forecasting in C.6).

- SARIMA forecasts exactly len(test) steps.
- Both are evaluated against the same ground truth vector.

3.3 Weighted ensemble and why weights matter

Instead of assuming 0.5 always works, I experimented with w_dl = 0.2, 0.5, 0.8:

```
def ensemble_weighted(pred_dl, pred_sarima, w_dl = 0.2):
```

- If SARIMA is worse than DL, a higher w_dl usually improves ensemble accuracy.
- These experiments demonstrate that ensemble performance is sensitive to weight selection.

3.4 Metrics used for comparison

Since the output is a continuous price, I evaluated models using:

- MAE (interpretable in price units)
- MSE and RMSE (penalize larger errors more).

```
  return get_prediction_index(
     DL  epochs  batch SARIMA_order SARIMA_seasonal  w_dl    DL_MAE  SARIMA_MAE   ENS_MAE  DL_time_s  SARIMA_fit_s
4   GRU      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.8  2.461807    8.885275  3.564750  14.140733      0.355737
1   GRU      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  3.140391    8.885275  5.691527  14.074919      0.347952
2  LSTM      25     64    (2, 1, 2)    (1, 0, 1, 5)   0.5  3.719337    8.888413  6.266070  12.391959      0.872914
0  LSTM      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  4.355780    8.885275  6.374318  12.522721      0.366028
3   RNN      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  4.932236    8.885275  6.891290   6.253570      0.321122
5   GRU      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.2  2.973884    8.885275  7.561548  18.235186      0.344126
Best config: {'DL': 'GRU', 'epochs': 25, 'batch': 64, 'SARIMA_order': (1, 1, 1), 'SARIMA_seasonal': (1, 0, 1, 5), 'w_dl'
: 0.8, 'DL_MAE': 2.4618070851201583, 'SARIMA_MAE': 8.885274888443167, 'ENS_MAE': 3.5647500933194056, 'DL_time_s': 14.140
733000007458, 'SARIMA_fit_s': 0.3557366000022739}
```

4. References Used

- *statsmodels SARIMAX / SARIMA documentation: https://www.statsmodels.org/stable/generated/statsmodels.tsa.statespace.sarimax.SARIMAX.html*
- *ARIMA/SARIMA concepts (p,d,q and seasonal P,D,Q,m): https://www.statsmodels.org/stable/tsa.html*
- *Keras recurrent layers (LSTM/GRU/RNN behavior, return_sequences): https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM*

  https://www.tensorflow.org/api_docs/python/tf/keras/layers/GRU

  https://www.tensorflow.org/api_docs/python/tf/keras/layers/SimpleRNN

5 Experiment Results Summary (Ensemble configurations & weights)

5.1 Different ensemble configurations (DL type + SARIMA params)

I tested ensemble configurations using:

- DL types: **LSTM / GRU / RNN**
- SARIMA orders: (1,1,1) and (2,1,2)
- seasonal order fixed to (1,0,1,5)

- ensemble weights: w_dl varied

```
  return get_prediction_index(
     DL  epochs  batch SARIMA_order SARIMA_seasonal  w_dl    DL_MAE  SARIMA_MAE   ENS_MAE  DL_time_s  SARIMA_fit_s
4   GRU      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.8  1.617517    8.885275  2.792504  14.006387      0.350979
1   GRU      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  1.748000    8.885275  5.160743  14.359247      0.335418
3   RNN      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  4.598762    8.885275  6.711254   6.338109      0.336550
0  LSTM      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  5.653851    8.885275  6.869697  13.909196      0.363868
2  LSTM      25     64    (2, 1, 2)    (1, 0, 1, 5)   0.5  5.936951    8.888413  6.984437  12.312003      0.892127
5   GRU      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.2  2.848655    8.885275  7.528944  14.411344      0.334484
Best config: {'DL': 'GRU', 'epochs': 25, 'batch': 64, 'SARIMA_order': (1, 1, 1), 'SARIMA_seasonal': (1, 0, 1, 5), 'w_dl'
: 0.8, 'DL_MAE': 1.617516990329908, 'SARIMA_MAE': 8.885274888443167, 'ENS_MAE': 2.7925044958638 52, 'DL_time_s': 14.00638
7099972926, 'SARIMA_fit_s': 0.35097899998072535}
```

```
  return get_prediction_index(
     DL  epochs  batch SARIMA_order SARIMA_seasonal  w_dl    DL_MAE  SARIMA_MAE   ENS_MAE  DL_time_s  SARIMA_fit_s
1   GRU      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  1.899075    8.885275  5.187256  14.271732      0.324552
3   RNN      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  3.791990    8.885275  5.584152   6.337987      0.368850
0  LSTM      25     64    (1, 1, 1)    (1, 0, 1, 5)   0.5  3.550960    8.885275  6.094776  13.779555      0.339619
2  LSTM      25     64    (2, 1, 2)    (1, 0, 1, 5)   0.5  4.247310    8.888413  6.286767  12.380923      0.895533
Best config: {'DL': 'GRU', 'epochs': 25, 'batch': 64, 'SARIMA_order': (1, 1, 1), 'SARIMA_seasonal': (1, 0, 1, 5), 'w_dl'
: 0.5, 'DL_MAE': 1.8990751515264086, 'SARIMA_MAE': 8.885274888443167, 'ENS_MAE': 5.1872564563632695, 'DL_time_s': 14.271
732299996074, 'SARIMA_fit_s': 0.3245524999802001}
```

5.2 Best-performing configurations (summary)

Key observations:

- SARIMA alone produced much higher error (MAE ≈ 8.885), so DL dominated performance.
- GRU produced the lowest DL_MAE among tested DL models.
- Increasing w_dl improved ensemble MAE because SARIMA was weaker than DL.

5.3 Comparison table (Best config per weight run)

A	B	C	D	E	F	G	H	I	J
Run label	**Best DL**	**SARIMA order**	**Seasonal**	**w_dl**	**DL_MAE**	**SARIMA_MAE**	**ENS_MAE**	**DL_time (s)**	**SARIMA_fit (s)**
w_dl = 0.5 (default run)	GRU	(1,1,1)	(1,0,1,5)	0.5	2.559094	8.885275	5.47199	14.160172	0.319661
w_dl = 0.8 run	GRU	(1,1,1)	(1,0,1,5)	0.8	1.873776	8.885275	3.119826	14.604878	0.314568
"w_dl = 0.2 run" (your best row is still w=0.8)	GRU	(1,1,1)	(1,0,1,5)	0.8	1.617517	8.885275	2.792504	14.006387	0.350979

Final best ensemble (lowest ENS_MAE):

- DL: GRU
- SARIMA: (1,1,1) seasonal (1,0,1,5)
- w_dl = 0.8
- ENS_MAE ≈ 2.7925

6. Conclusion

Task C.6 successfully implemented ensemble forecasting by combining a statistical model (SARIMA) with deep learning models (LSTM/GRU/RNN). Experiments show that ensemble performance depends strongly on the ensemble weight and the relative accuracy of each component model. In my experiments, GRU was the strongest DL model, and increasing w_dl improved ensemble results because SARIMA had higher error.Close).